

Fruitfulness as Compression: An Exact Calculus of Naming, a Speed Limit, and a Measurement of All of Mathlib

Vasily Shablov*

October 2026

Draft

Abstract

We pose an open-ended question: what is the *fruitfulness* of a mathematical idea? Our answer is that a named concept is a compression, and its value is what naming it saves. We develop this into a theory, prove its theorems in Lean 4 (about 1,100 lines, no `sorry`), and measure it on all 771,129 constants of Mathlib.

An exact calculus of naming.

- In any library, naming one more result v saves exactly $(o - 1)(W - 1) - 1$ symbols. Here o is how often v occurs and W is how much it hides.
- Naming changes all inlined sizes and occurrence counts by *rank-one updates*: the Schur complements of Gaussian elimination. Inlining a concept is eliminating a variable.
- When every result is used, the savings function is *submodular*: concepts are substitutes, never complements, as far as compression goes.
- For a reader of bounded capacity, by contrast, names can be strict complements. We conclude that the increasing returns of great concepts are a phenomenon of bounded minds, not of compression.

A speed limit.

- Naming can make mathematics exponentially shorter, but only at a bounded rate. Every unfolded size is at most $28 \cdot (5/4)^C$, where C is the named size of the library. The proof is a potential-function argument with weights found by linear programming.
- Explicit Fibonacci and Lucas libraries reach $\varphi^{C/3}$.
- We conjecture that the golden rate $\varphi^{1/3}$ is exact. At named size $3m$ the maximum is $2L_m - 2$, with L_m the Lucas numbers. This is proved attainable, and verified to be optimal for $4 \leq m \leq 15$.

Measurements.

- Unfolded size grows about $1.5\times$ per dependency level ($R^2 = 0.98$).
- Reuse alone rediscovers the textbook core (`Set`, `Category`, `Functor`, ...).
- Human taste splits along exactly the two factors of the exact value: concepts by occurrence (AUC 0.88), famous theorems by depth (AUC 0.78). Humans do not, however, value things by the product of the two (AUC 0.62).
- A structural glue layer takes 12% of constants but 60% of citations, the signature of function words and currency metabolites.

We correct an overreach in our own earlier statement and connect the work to proof complexity, Gaussian elimination and renormalization, logical depth, compression progress, and hierarchy in nature. We state open questions and falsifiable conjectures.

*Formalization, measurement and drafting assisted by Claude (Anthropic).

1 Introduction

Where the question came from. The question started as a debate. One position holds that AI is good at well-posed yes/no problems but cannot handle open “why / what is” questions, because “there is nothing to optimize.” Rather than argue, we posed one such question and followed it:

What is the fruitfulness of a mathematical idea, and can it be measured?

Our answer. We take a compression view, in the tradition of minimum description length and of Whitehead’s remark that “civilization advances by extending the number of important operations which we can perform without thinking about them.”

- A definition or lemma is *named*: its body is written once and then cited.
- The value of a name is the description length it saves.
- This makes fruitfulness precise enough to prove theorems about, and concrete enough to measure on the largest library of formalized mathematics.

Contributions.

1. **Single-name accounting** (Section 2). The value $(k - 1)(b - 1) - 1$, tight exponential bounds on unfolding, and a sharp d -bonacci bound.
2. **A calculus of naming** (Section 3). The exact value of a name in any library, rank-one (Schur) update rules, submodularity (concepts are substitutes), and complementarity for bounded readers.
3. **A speed limit** (Section 4). A proved bound of $(5/4)^C$, golden constructions, and the Lucas conjecture.
4. **A complete measurement of Mathlib** (Sections 5–6), including a test of the calculus against human taste.
5. **Connections and open questions** (Section 7).

All theorems are machine-checked: `Fruitfulness.lean`, `Naming.lean` and `SpeedLimit.lean` (Appendix A).

2 Theory: what naming buys

Definition 1 (Library). A library *consists of*:

- a *size function* $\text{size} : \mathbb{N} \rightarrow \mathbb{N}$;
- *citation sets* $\text{deps}(i) \subseteq \{0, \dots, i - 1\}$.

Result i has its own proof of size $\text{size}(i)$ and cites the results in $\text{deps}(i)$. Its unfolded size is

$$U(i) = \text{size}(i) + \sum_{j \in \text{deps}(i)} U(j),$$

which is the size of its proof if each cited result were re-proved in place.

Theorem 1 (Value of one name; `naming_saves`, `naming_pays_iff`). *Inlining a body of size b at k sites costs kb . Naming it costs $b + k$: one definition plus one reference per use. The saving is exactly*

$$kb - (b + k) = (k - 1)(b - 1) - 1.$$

- *Naming pays off if and only if $(k - 1)(b - 1) > 1$. In particular it always pays when $k \geq 2$ and $b \geq 3$ (`naming_pays_of`).*

- A name used once never pays: it costs exactly one symbol (*naming_once*).

The last clause is the *rule utility* constraint of the Sequitur grammar compressor [21], here derived rather than imposed.

Theorem 2 (Doubling; `Library.unfold_le_two_pow`, `completeLib_unfold`). *If $\text{size} \leq S$, then $U(i) \leq S \cdot 2^i$, whatever the number of citations. Equality holds for the complete library, in which every result cites all earlier ones and has size 1: there $U(i) = 2^i$.*

Theorem 3 (Sharp bound for bounded citation; `Library.unfold_le_tb`, `windowLib_unfold`). *Let $T_d(i) = 1 + \sum_{j=\max(0, i-d)}^{i-1} T_d(j)$. If $\text{size} \leq S$ and $|\text{deps}(i)| \leq d$ for all i , then*

$$U(i) \leq S \cdot T_d(i).$$

Equality holds for the library in which each result cites its d predecessors and has size 1.

The proof rests on an exchange lemma (`sum_le_sum_window`): among sets of at most d indices below i , a monotone function has its largest sum on the last d indices. Some consequences:

- $T_d(i)$ grows like ϕ_d^i , where ϕ_d is the d -bonacci constant, the root in $(1, 2)$ of $x^d = x^{d-1} + \dots + 1$.
- ϕ_2 is the golden ratio. The case $d = 2$ is formalized separately: $U(i) = F_{i+3} - 1 \geq 2^{\lfloor i/2 \rfloor}$ (`fibLib_unfold`, `fibLib_unfold_ge`).
- $\phi_d \rightarrow 2$ as $d \rightarrow \infty$, recovering the doubling theorem.

Every one of these exponential gaps is generated by a library of *linear* named size.

Remark (What these theorems do and do not say). U measures *copying*. It is not the size of the shortest lemma-free proof, and a clever re-prover can restructure a proof rather than inline it. For Frege systems, tree-like proofs polynomially simulate dag-like ones [17]. So, in that setting, reusing lemmas saves only polynomially, even though copying costs exponentially. The question of whether *new symbols* (definitions) can save super-polynomially is the Frege versus Extended Frege problem, which is open (Section 7). Our theorems are about naming versus copying, and they are tight for that.

3 A calculus of naming

Section 2 priced one name in isolation. Real libraries have names inside names, and the value of one name depends on every other naming decision. Here we give an exact theory of that dependence.

Setting.

- Results $0, 1, 2, \dots$ have own sizes $\text{size}(x) \geq 1$ and citations $\text{deps}(x)$ of smaller index.
- A *naming* is a finite set N of results. A named result is written once and cited by a one-symbol reference. Every other result is inlined wherever it occurs.
- A goal r is always written.

Define, by recursion over the DAG:

$$W_N(x) = \text{size}(x) + \sum_{j \in \text{deps}(x)} \begin{cases} 1 & j \in N \\ W_N(j) & j \notin N \end{cases} \quad (\text{inlined size of } x)$$

$$P_N(x, v) = \sum_{j \in \text{deps}(x)} \begin{cases} 1 & j = v \\ 0 & j \in N \\ P_N(j, v) & \text{else} \end{cases} \quad (\text{occurrences of } v \text{ in } x)$$

- $\text{cost}(N) = W_N(r) + \sum_{x \in N} W_N(x)$ is the written size.
- $o_N(v) = P_N(r, v) + \sum_{x \in N} P_N(x, v)$ is the number of occurrences of v in the written library.

Theorem 4 (Exact value of a name; `Lib.gain_eq`). *For $v \notin N$,*

$$\text{cost}(N) - \text{cost}(N \cup \{v\}) = (o_N(v) - 1)(W_N(v) - 1) - 1.$$

The value of a concept is a product of two factors:

- an *external* one: how often the concept is needed;
- an *internal* one: how much it hides.

Section 6 finds that human taste splits along exactly these two factors.

Theorem 5 (Rank-one updates; `Lib.W_insert`, `Lib.P_insert`, `Lib.occ_insert`). *Let $u \notin N$ and $N' = N \cup \{u\}$. For all x and all $v \neq u$:*

$$\begin{aligned} W_{N'}(x) &= W_N(x) - P_N(x, u) (W_N(u) - 1), \\ P_{N'}(x, v) &= P_N(x, v) - P_N(x, u) P_N(u, v), \\ o_{N'}(v) &= o_N(v) - (o_N(u) - 1) P_N(u, v). \end{aligned}$$

(In Lean these are stated without subtraction, as identities in $\mathbb{N}$.)

Naming is pivoting. Write A for the citation matrix and D for the diagonal projection onto unnamed results. Occurrence counts are then the entries of $(I - AD)^{-1}A$: paths whose interior avoids names. Naming u removes u from D , and Theorem 5 is exactly the Sherman–Morrison rank-one update of this inverse. It is the Schur complement of Gaussian elimination, read in reverse. So:

- **Inlining a concept is eliminating a variable.** In electrical networks this is Kron reduction, and in physics it is integrating out a degree of freedom, the elementary step of renormalization.
- **Naming a concept is keeping it as an interface.**

The analogy has teeth. Choosing an elimination order that minimizes fill-in is NP-complete [4]. We ask below whether optimal naming is too.

Theorem 6 (Concepts are substitutes; `Lib.substitutes`). *Suppose every result below the goal is cited by some result at or below the goal. Then for all N and all distinct $u, v \notin N$ below the goal,*

$$\text{gain}_{N \cup \{u\}}(v) \leq \text{gain}_N(v).$$

Equivalently, $N \mapsto \text{cost}(\emptyset) - \text{cost}(N)$ is submodular.

Proof sketch. The proof combines three facts.

- By Theorem 5, naming u can only lower $W(v)$ (since $W \geq 1$).
- It can only lower $o(v)$, provided u itself occurs (`Lib.occ_insert_le`).
- A downward induction shows that every used result always occurs at least once, whatever is named (`Lib.one_le_occ`).

Both factors of Theorem 4 are then nonnegative and non-increasing. □

Consequences. Submodularity is the structure behind the greedy and double-greedy guarantees of combinatorial optimization [3, 2]. More striking is what it says about mathematics. *For compression of a fixed body of mathematics, every new concept is worth less the more concepts we already have.* Concepts never make each other more valuable. Yet the history of mathematics is full of the opposite: a concept that makes others fruitful, a definition that opens a field.

Theorem 7 (Bounded readers see complements; `comprehension_complements`). *There is a library with two lemmas and a goal such that, for a reader who can hold at most 3 symbols:*

- *the goal is unreadable with no names, and with either name alone;*
- *it is readable with both names.*

Comprehension, unlike compression, is not submodular.

Taken together, Theorems 6 and 7 locate the source of increasing returns. It is not compression. It is *bounded capacity*: a reader who must fit a statement into working memory needs several concepts at once, and each is worthless alone. This holds for any bounded reader, human or machine. It formalizes Thurston’s insistence that mathematics is about understanding [25], and explains why that insistence matters.

4 The speed limit of compression

How much can naming buy? Section 2 bounds unfolding per result. The right currency, though, is the *named size* of a library: the total number of symbols written down.

Problem. Each result has unit own size and cites distinct earlier results. Its unfolded size $u_i = 1 + \sum_{j \in \text{deps}(i)} u_j$ is the number of citation paths starting at i . The named cost of the first n results is $C = \sum_{i < n} (1 + |\text{deps}(i)|)$: one symbol per result and one per citation. What is the largest possible u_i for a given C ?

Theorem 8 (Speed limit; `ULib.u_le`). *Every unfolded size satisfies*

$$u_i \leq 28 \cdot (5/4)^C$$

(in Lean: $4^C u_i \leq 28 \cdot 5^C$).

Proof idea. Let S_j be the largest total unfolded size of at most j distinct results so far: a “top- j sum”. The potential

$$\Phi = 4S_1 + 2S_2 + S_3 + 28$$

grows by at most a factor $(5/4)^{1+d}$ when a result citing d others is added.

- The weights $(4, 2, 1)$ were found by linear programming.
- The required inequalities follow from three elementary properties of top- j sums: monotonicity, subadditivity $S_{a+b} \leq S_a + S_b$, and the averaging bound $jS_{j+1} \leq (j+1)S_j$.
- For $d \leq 9$, Lean’s `omega` finds each certificate (`key_small`). For $d \geq 10$ an explicit exponential estimate suffices.

□

Theorem 9 (Golden constructions; `fibLib_u`, `fibLib_cost`, `lucasLib_top`, `lucasLib_cost`).

- **Fibonacci library.** *If each result cites its two predecessors, then at cost $3(n+1)$ the top result has unfolded size $F_{n+3} - 1$.*
- **Lucas library.** *Take a leaf, a two-step chain, a Fibonacci core, and a final result citing the top three. For $m \geq 4$ this reaches exactly $2L_m - 2$ at cost $3m$.*

So the optimal rate per symbol lies between $\varphi^{1/3} \approx 1.174$ and $5/4$. The trivial product bound gives only $3^{1/3} \approx 1.442$.

Exact search. The state of a library is the multiset of its unfolded sizes, and citing the largest available sizes is always optimal. This permits an exact search over Pareto-optimal states (within a citation window). It gives:

m	4	5	6	7	8	9	10	11	12	13	14	15
max at cost $3m$	12	20	34	56	92	150	244	396	642	1040	1684	2726
$2L_m - 2$	12	20	34	56	92	150	244	396	642	1040	1684	2726

These values are exact for windows of up to 9 citations (cost ≤ 39) and up to 5 citations (cost ≤ 45).

Conjecture 1 (Lucas law of compression). *For every $m \geq 4$, the largest unfolded size of a library of named cost $3m$ is exactly $2L_m - 2$. In particular, the speed limit of compression is $\varphi^{1/3}$ per symbol.*

Why the golden ratio, and why the proof is hard.

- **The optimal citation count is two.** Citing d predecessors costs $d + 1$ per result and grows like the d -bonacci constant φ_d . Per symbol, $\varphi_d^{1/(d+1)}$ is largest at $d = 2$: 1.174 against 1.165 for $d = 3$. Two is the “optimal radix” of mathematical citation.
- **Linear potentials provably cannot reach the conjecture.** By LP duality, no linear potential in top- j sums certifies a rate below 1.2198. A piecewise-linear potential built from Fibonacci continuations fails on states holding many copies of their largest value. This suggests the true value function has infinitely many linear pieces.
- **A kindred conjecture.** The golden ratio is also the conjectured growth rate of monotone paths on simple 3-polytopes [1], where the best upper bounds likewise exceed φ .

Bounded readers reach exponentially far. In the Fibonacci library every result has three symbols: itself and two citations (`fibLib_readable`). A reader who can hold three symbols can therefore check every step, yet the results reach unfolded size $\varphi^{C/3}$. No library can push this past $(5/4)^C$. Understanding scales exponentially with the library, and at a bounded rate.

5 Data and method

Export. We loaded all of Mathlib [19] (Lean v4.33.1), including private proof bodies, and recorded every constant: its kind, its module, the size of its body (number of distinct term nodes), and the constants cited in its body and statement. In a second pass, with environment extensions loaded, we recorded the size of its statement and whether it is an instance declaration or a structure projection.

Size of the data.

- 771,129 constants and 20.4M citation edges.
- 279 cycle edges, from unsafe definitions, were dropped.
- The *visible* set has 315,085 constants: user-facing Mathlib theorems, definitions and inductive types, after removing compiler-generated auxiliaries.

Measures. For each constant c :

- *reuse* $k(c)$: the number of constants citing c ;
- *unfolded size* $U(c)$: as in Section 2, computed exactly in log-space;

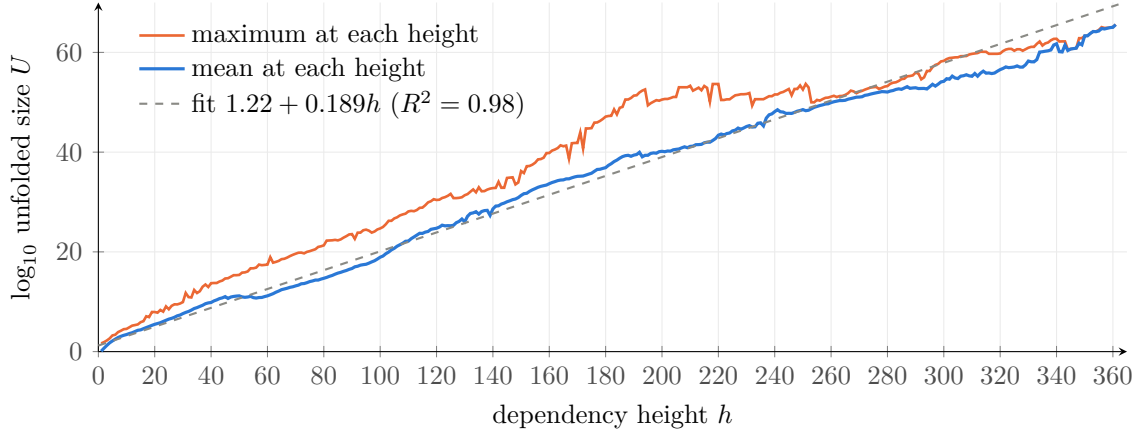


Figure 1: Naively unfolded size grows exponentially with dependency height, about $1.5\times$ per level, over 315,085 Mathlib constants.

- *height*: the longest citation chain below c ;
- *depth*: $\log_{10} U(c) - \log_{10}(\text{statement size})$, i.e. hidden work per symbol of statement.

Glue. We call instance declarations and structure projections *glue*, classified structurally rather than by name. They make up 12.3% of visible constants.

Human labels. All labels come from Mathlib’s own human-curated files.

- *Famous*: Freek Wiedijk’s “100 theorems” and the “1000+ theorems” project, 294 declarations.
- *Curriculum*: the undergraduate curriculum list and the library overview, 686 declarations.

Statistics.

- AUC against all other non-glue constants of the same kind. 0.5 is chance and 1.0 is perfect.
- A *same-file* control: the probability that a labelled constant beats an unlabelled constant of the same kind in the same source file. This controls for “advanced files contain deeper things.”

6 Results

6.1 A growth law

Mean $\log_{10} U$ is almost exactly linear in height (Figure 1).

- **Fit.** Over all 315,085 constants, $\log_{10} U \approx 1.22 + 0.189 h$ with $R^2 = 0.979$, a factor of 1.55 per level. Fitting the per-height means gives 1.50 per level with $R^2 = 0.990$.
- **Stability across the range.** The three thirds of the height range give 1.45, 1.64 and 1.41.
- **Placement.** This is supercritical branching: well above the linear regime, and below the golden ratio, 1.618 per *result*, of the citing-two-predecessors library.
- **Scale.** The deepest result, `CompletelyPositiveMap.map_cstarMatrix_nonneg` at height 361, has $U \approx 10^{65.6}$.

6.2 Reuse rediscovers the textbook

With glue removed, the most-cited definitions and theorems are listed below. No notion of importance was supplied. The ranking is pure citation counting.

Human label	n	reuse k	unfolded U	depth	statement size
curriculum <i>definitions</i>	421	0.88 / 0.87	0.46 / 0.35	0.46 / 0.38	0.36 / 0.32
curriculum theorems	209	0.58 / 0.59	0.82 / 0.66	0.82 / 0.68	0.67 / 0.51
famous theorems	259	0.54 / 0.55	0.78 / 0.66	0.78 / 0.67	0.59 / 0.51

Table 1: Global AUC / same-file win rate, glue removed. 0.5 is chance.

Definitions (by citations)	Theorems (by citations)
Set, Category, Functor, TopologicalSpace, Real, Module, RingHom.id, CommRing, Finset, Semiring, RingHom, AddCommMonoid, AddCommGroup, Algebra, NormedAddCommGroup, LinearMap, Equiv, Opposite, ...	mul_one, add_zero, le_trans, zero_add, one_mul, Nat.cast_one, mul_comm, Finset.sum_congr, Set.ext, le_refl, mul_assoc, map_zero, le_antisymm, ...

The theorem column omits tactic-internal certificate lemmas (`Mathlib.Tactic.*`), which form a third, automation-generated kind of glue.

Observation. The top of the concept ranking is dominated by notions of *structure and sameness*: sets, maps, homomorphisms, equivalences, categories, functors, opposites. The single most-cited definition overall is `DFunLike.coe` (applying a structured map), at 115,373 citations. In a precise sense, the most compressive idea in modern mathematics is the morphism.

6.3 Two axes of human taste

Table 1 gives the main result.

- **Concepts.** Human-curated concepts are predicted by *reuse*: AUC 0.88, and 0.87 against peers in the same file. Depth does not predict them, and is even inverted (AUC 0.46): fundamental concepts are shallow.
- **Theorems.** Famous and curriculum theorems are predicted by *depth*: AUC 0.78–0.82, and 0.66–0.68 within the same file. Reuse barely predicts them (0.54–0.58).
- **Famous theorems are not reuse hubs.** 28.6% of famous declarations are never cited anywhere else in Mathlib, against 36.6% of all theorems, so they are only slightly more reused than average.

The product is not what humans use. Theorem 4 says the compression value of a concept is the product of occurrence and hidden size. We tested whether human labels, pooled over concepts and theorems (841 labelled constants, glue removed), track the product. They do not.

- Reuse alone: AUC 0.73.
- Depth alone: AUC 0.59.
- The product, $\log k + \log U$: AUC 0.62.

Humans use each factor separately, by kind: occurrence for concepts, depth for theorems. They do not use the compression value itself. Taste is not compression.

6.4 A glue layer, a heavy tail, and the law of abbreviation

Glue. Glue (instances and projections) is 12.3% of visible constants but receives 59.5% of all citations, including 77 of the 100 most-cited constants.

- Only 63 of the 904 human-labelled declarations are glue.

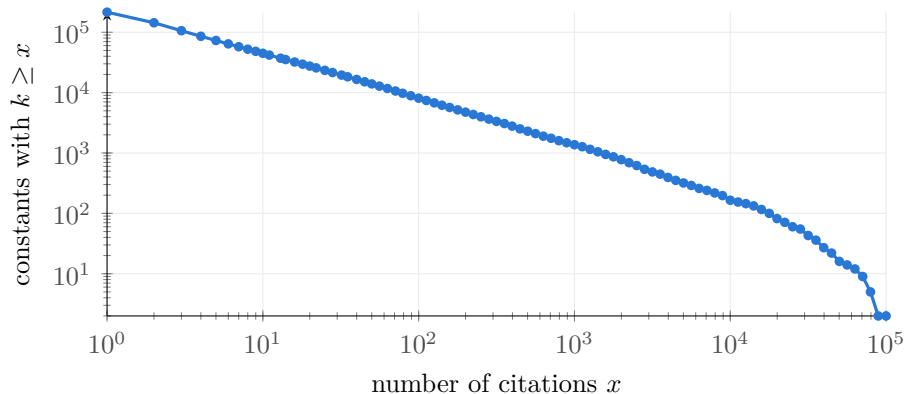


Figure 2: Reuse in Mathlib is heavy-tailed: 215,078 constants are cited at least once, 8,090 at least 100 times, and 165 at least 10,000 times.

- This is the profile of *function words* in language: a small closed class carrying a large share of tokens and little content.
- It is also the profile of *currency metabolites* (ATP, NADH, water), which network biologists must remove before metabolic pathways become visible [18].

Heavy tail. Reuse is heavy-tailed (Figure 2).

- A discrete power-law maximum-likelihood fit [11] gives a density exponent $\alpha \approx 1.72$, 1.77 and 1.79 for $k \geq 10$, 30 and 100.
- For comparison: word frequencies give $\alpha \approx 2$ (Zipf), and protein-domain family sizes and citation counts show similar tails [16, 22].

Law of abbreviation. In natural languages, frequent words are shorter (Zipf), a regularity proposed to be a universal consequence of compression [14]. Mathlib obeys it too.

- **Definitions.** Mean name length falls monotonically with reuse: 13.2 characters (never cited), 10.8 (cited 10–99 times), 8.9 (10^3 – 10^4), 8.2 (10^4 – 10^5), and 3 for **Set**.
- **Theorems.** Mean name length falls from 19.2 to 8.5.
- **Rank correlations.** Spearman $\rho(k, \text{name length}) = -0.16$ for definitions and -0.07 for theorems; $\rho(k, \text{body size}) = -0.22$ for definitions. Most constants have small k , so the global correlations are weak, but the trend across decades of k is monotone.

Mathlib’s names are derived from statements by convention, so this is not designed; it emerges from which concepts get reused.

7 Connections and open questions

7.1 Are concepts necessary? Proof complexity

Naming comes in two strengths.

- **Lemmas** reuse proved statements: dag-like versus tree-like proofs.
 - The gap is exponential for resolution [5].
 - It is polynomial for Frege [17].
- **Definitions** introduce new symbols. This is Tseitin’s extension rule; Frege plus extension is Extended Frege [12].

– Whether Extended Frege is super-polynomially stronger than Frege is a central open problem. Higher up, the answer is known to be dramatic. Gödel [15] showed that passing to higher-order logic shortens some proofs by more than any computable function, and Boolos’s “curious inference” [7] gives a concrete first-order-valid argument whose first-order proofs are physically unwritable but which has a short second-order proof.

Question 1 (Are new concepts forced?). Are *definitions* ever super-polynomially necessary beyond lemma reuse? This is Frege versus Extended Frege, restated as the question “are human concepts forced or merely convenient?”

Question 2 (Empirical proof complexity). Mathlib is a vast sample of definitions in actual use. Can it locate *where* definitions give large savings, and so supply empirical evidence on a problem that has resisted proof?

7.2 Value has two axes: depth and leverage

The split in Table 1 matches two independent theories of value.

- **Bennett’s logical depth** [6]. An object is valuable when it is the product of a long computation that cannot be shortcut. Famous theorems are deep: a short statement certifies a large amount of hidden work.
- **Schmidhuber’s compression progress** [23]. Interest equals improvement of the observer’s compressor. Our $(k - 1)(b - 1) - 1$ is exactly the compression progress of one new concept, and reuse predicts concept taste.

We know of no previous large-scale test of either thesis against human judgments of mathematical value, though a fuller literature search is warranted.

Question 3 (The residual). Are depth and leverage the only two axes of mathematical value? What explains the remaining 12–22% of AUC: surprise, beauty, *bridging* between distant areas, or historical accident?

Question 4 (Computability of taste). Ideal fruitfulness, meaning the optimal library of concepts for all of mathematics, is a minimal-grammar problem. It is NP-hard even for finite corpora [9], and in the limit it is Kolmogorov complexity, which is uncomputable. So no agent, human or machine, can optimize fruitfulness exactly; all can only approximate it from below. Is the human advantage in choosing directions, if any, then a matter of better approximation rather than a different kind of faculty?

7.3 A glue law across nature

Function words, currency metabolites, tiny utility packages (the `left-pad` incident), and Mathlib’s instances share a profile: few units, an enormous share of uses, little content, and invisible to anyone asked what matters.

Question 5 (The glue theorem). Must every system that grows by reuse under a description-length cost develop (i) heavy-tailed reuse, (ii) a high-frequency, low-content glue layer, and (iii) abbreviation of the frequent? Preferential attachment [22] explains (i), and compression arguments [14] explain (iii). Is there a single derivation of all three?

Mathlib is a natural *model organism* for such laws.

- In biology, edges are inferred and noisy. In language, “concept” is fuzzy.
- In Mathlib, every edge is exact and every unit is machine-checked.

Question 6 (Mathematics as an evolving genome). Protein-domain statistics are explained by birth–death–innovation models [16]. Does the same model, fitted to Mathlib’s history (generalization as duplication, imported concepts as horizontal transfer), reproduce its reuse distribution?

7.4 Hierarchy, decoupling, and the effectiveness of mathematics

Simon’s watchmakers. Simon’s parable [24] contrasts a watchmaker who builds from stable sub-assemblies with one who does not. The first has an exponential advantage under interruption, which Simon offered as the reason nature is hierarchical. Our theorems are the corresponding statement about *size*.

Proof irrelevance is scale separation. In Lean, any two proofs of the same proposition are equal. Later results therefore depend only on *statements*, never on proofs. This is the logical form of the physicist’s decoupling of scales: chemistry needs the periodic table, not quantum chromodynamics. The depth $\log U - \log(\text{statement})$ measures how much micro-detail a statement forgets.

Question 7 (Wigner’s puzzle as compression). Is the “unreasonable effectiveness” of mathematics in physics [26] a consequence of both being hierarchical compressions under scale separation? Formalized physics libraries now exist. Do they reuse the same top concepts as Mathlib, with the same tail exponent and glue layer?

7.5 Delayed reward and understanding

Question 8 (How delayed is the reward?). The strongest form of the “nothing to optimize” objection is that fruitfulness is visible only decades later; Lobachevsky is the canonical example. Mathlib’s version history records when each declaration and each citation appeared. Does early reuse, after months, predict late reuse, after years? If it does, the reward horizon is short and learnable. If it does not, the objection gains real empirical support.

Question 9 (Understanding as conditional compression). Thurston held that mathematics is about human understanding [25]. Define: an agent *understands* a proof when its size, relative to the agent’s library of named concepts, fits within the agent’s working capacity. This is conditional description length with a library in place of a universal machine, consistent with chunking in cognition [10, 20]. On this definition, “can humans understand machine mathematics?” becomes the quantitative question “can the machine’s library be taught?”

7.6 Questions raised by the calculus

Question 10 (Complexity of optimal naming). Is choosing the optimal naming N NP-hard in the model of Section 3, as minimal fill-in is for elimination orderings [4]? By Theorem 6 the savings function is submodular. When it is also nonnegative, randomized double greedy gives a $1/2$ -approximation [2]. Is there a better algorithm that exploits the rank-one structure?

Question 11 (Where do increasing returns live?). Theorem 7 shows that bounded capacity creates complementarity.

- Is every complementarity between concepts in real mathematics explained by capacity?
- Or does some come from the *growth* of the body of theorems (new statements become expressible), which a fixed-corpus model cannot see?
- Can Mathlib’s history separate these two sources?

Question 12 (Renormalization of mathematics). Inlining is elimination, and elimination is integrating out. Is there a renormalization flow on libraries, with fixed points, under which Mathlib’s 1.5×-per-level growth (Section 6) is a scaling exponent?

Question 13 (Speed limit with general sizes). What is the speed limit when own sizes vary, or when citation multiplicities are allowed (a result using a lemma twice)? With multiplicities, doubling is free, and the limit should approach $3^{1/3}$, the optimal-radix constant of the product bound. Is the transition between $\varphi^{1/3}$ and $3^{1/3}$ sharp?

7.7 Falsifiable conjectures

Conjecture 2 (Growth law). *In future versions of Mathlib, the per-level unfolding factor stays in $[1.4, 1.7]$ with $R^2 > 0.95$.*

Conjecture 3 (Two axes). *In other large formal libraries (Isabelle AFP, Rocq/MathComp), human-curated concepts are predicted by reuse and famous theorems by depth, each with $AUC > 0.75$. Reuse does not predict fame ($AUC < 0.6$).*

Conjecture 4 (Glue share). *In any mature formal library with typeclass-style inference, a structurally defined glue class of at most 15% of constants receives a majority of citations.*

Conjecture 5 (Concepts are rediscovered). *Strip names from Mathlib’s proof terms and run compression-driven library learning [13, 8]. The highest-value recovered abstractions substantially overlap the top concepts of Section 6.*

Conjecture 6 (Morphisms are the universal compressor). *In every sufficiently large formal library, the most-cited definitions are dominated by notions of map and sameness (morphism, homomorphism, equivalence, functor) rather than by specific objects.*

8 Limitations

- **The labels are biased.** They record what formalizers chose to curate.
- **U is a copying measure.** It counts each citation once per citing result. It is not an occurrence count inside proof terms, and not a lower bound on proof size.
- **The glue classification is incomplete.** It is structural but leaves out tactic-certificate lemmas, which we report separately.
- **One snapshot.** All measurements come from a single version of Mathlib. Temporal claims are stated as conjectures.
- **The reported correlations are observational.**

9 Conclusion

An open “what is” question led to:

- a definition, and an exact calculus for it (Theorems 4–7);
- a proved speed limit and a golden conjecture with Lucas-number structure;
- a growth law over 315,085 objects;
- a split of human taste into exactly the two factors the calculus predicts, though humans do not use their product;
- connections from Gaussian elimination to renormalization, and from function words to currency metabolites.

Two conclusions stand out.

- **Concepts are substitutes for compression, but complements for understanding.** The explosive fruitfulness of great ideas is a property of bounded minds, and so of every mind we know how to build.
- **Fruitfulness is not unmeasurable.** It is delayed, and in its ideal form uncomputable, for every kind of mind.

A Lean index

There are three files, all compiling against Mathlib with no `sorry`. Every theorem listed uses only `propext`, `Classical.choice` and `Quot.sound`.

Statement	Lean name
<i>Fruitfulness.lean: single names and unfolding</i>	
Saving $(k-1)(b-1)-1$; pays iff $(k-1)(b-1) > 1$	<code>naming_saves</code> , <code>naming_pays_iff</code>
Used once never pays	<code>naming_once</code>
$U(i) \leq S^{2^i}$; attained by the complete library	<code>Library.unfold_le_two_pow</code> , <code>completeLib_unfold</code>
Sharp d -bonacci bound; attained	<code>Library.unfold_le_tb</code> , <code>windowLib_unfold</code>
Fibonacci library $U(i) = F_{i+3} - 1$	<code>fibLib_unfold</code>
<i>Naming.lean: the calculus of naming</i>	
Exact value $(o-1)(W-1)-1$	<code>Lib.gain_eq</code>
Rank-one updates of W , P , o	<code>Lib.W_insert</code> , <code>Lib.P_insert</code> , <code>Lib.occ_insert</code>
Every used result occurs	<code>Lib.one_le_occ</code>
Concepts are substitutes (submodularity)	<code>Lib.substitutes</code>
Bounded readers see complements	<code>comprehension_complements</code>
<i>SpeedLimit.lean: the speed limit</i>	
Top- j sums: subadditive, averaging bound	<code>ULib.S_add</code> , <code>ULib.S_avg</code>
Potential step and speed limit $28(5/4)^C$	<code>ULib.Φ_step</code> , <code>ULib.u_le</code>
Fibonacci library at cost $3(n+1)$; three-symbol readability	<code>fibLib_u</code> , <code>fibLib_cost</code> , <code>fibLib_readable</code>
Lucas library reaches $2L_m - 2$ at cost $3m$	<code>lucasLib_top</code> , <code>lucasLib_cost</code>

Reproduction: `Export.lean` and `Flags.lean` export the graph; `analyze.py`, `robustness.py` and `paper_stats.py` compute every number in this paper.

References

- [1] C. A. Athanasiadis, J. A. De Loera, Z. Zhang. Enumerative problems for arborescences and monotone paths on polytope graphs. *J. Graph Theory* (2022).
- [2] N. Buchbinder, M. Feldman, J. Naor, R. Schwartz. A tight linear time $(1/2)$ -approximation for unconstrained submodular maximization. *FOCS* 2012.
- [3] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Math. Programming* 14 (1978).
- [4] M. Yannakakis. Computing the minimum fill-in is NP-complete. *SIAM J. Algebraic Discrete Methods* 2 (1981).

- [5] E. Ben-Sasson, R. Impagliazzo, A. Wigderson. Near optimal separation of tree-like and general resolution. *Combinatorica* 24 (2004).
- [6] C. H. Bennett. Logical depth and physical complexity. In *The Universal Turing Machine: A Half-Century Survey*, 1988.
- [7] G. Boolos. A curious inference. *J. Philosophical Logic* 16 (1987).
- [8] M. Bowers et al. Top-down synthesis for library learning. *POPL* 2023.
- [9] M. Charikar et al. The smallest grammar problem. *IEEE Trans. Inf. Theory* 51 (2005).
- [10] W. G. Chase, H. A. Simon. Perception in chess. *Cognitive Psychology* 4 (1973).
- [11] A. Clauset, C. R. Shalizi, M. E. J. Newman. Power-law distributions in empirical data. *SIAM Review* 51 (2009).
- [12] S. A. Cook, R. A. Reckhow. The relative efficiency of propositional proof systems. *J. Symbolic Logic* 44 (1979).
- [13] K. Ellis et al. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI* 2021.
- [14] R. Ferrer-i-Cancho et al. Compression as a universal principle of animal behavior. *Cognitive Science* 37 (2013).
- [15] K. Gödel. Über die Länge von Beweisen. *Ergebnisse eines math. Kolloquiums* 7 (1936).
- [16] E. V. Koonin, Y. I. Wolf, G. P. Karev. The structure of the protein universe and genome evolution. *Nature* 420 (2002).
- [17] J. Krajíček. *Bounded Arithmetic, Propositional Logic, and Complexity Theory*. Cambridge University Press, 1995.
- [18] H. Ma, A.-P. Zeng. Reconstruction of metabolic networks from genome data and analysis of their global structure. *Bioinformatics* 19 (2003).
- [19] The mathlib Community. The Lean mathematical library. *CPP* 2020.
- [20] G. A. Miller. The magical number seven, plus or minus two. *Psychological Review* 63 (1956).
- [21] C. G. Nevill-Manning, I. H. Witten. Identifying hierarchical structure in sequences: a linear-time algorithm. *JAIR* 7 (1997).
- [22] D. de Solla Price. A general theory of bibliometric and other cumulative advantage processes. *JASIS* 27 (1976).
- [23] J. Schmidhuber. Driven by compression progress. In *Anticipatory Behavior in Adaptive Learning Systems*, Springer, 2009.
- [24] H. A. Simon. The architecture of complexity. *Proc. Amer. Phil. Soc.* 106 (1962).
- [25] W. P. Thurston. On proof and progress in mathematics. *Bull. AMS* 30 (1994).
- [26] E. Wigner. The unreasonable effectiveness of mathematics in the natural sciences. *Comm. Pure Appl. Math.* 13 (1960).